

RSA Encryption, Signature, JWT Bypass, and Availability Demo

Technical Code Walkthrough

Codex Lab

2026-05-30

Contents

1. Overview	2
2. Files and Responsibilities	2
CryptoDemoApp.java	2
AttackClient.java	2
DashboardApp.java	3
DemoLog.java	3
DemoJson.java	3
3. RSA Refresher	3
4. Secure RSA in This Code	4
Key Generation	4
Secure Encryption: RSA-OAEP-SHA256	4
Secure Signature: RSASSA-PSS-SHA256	4
5. Toy RSA: How the Weak RSA Attack Works	5
Challenge Creation	5
Attacker Calculation	6
What This Demonstrates	6
6. JWT and Signature Bypass	6
Normal JWT Structure	6
Secure Verification	7
Vulnerable Bypass	7
Verification Policy	7
7. DDOS / Availability Load Demo	7
What This Demonstrates	8
8. Dashboard and Logs	8
9. Why Old Metrics Reappear After Restart	9
10. How To Reset Dashboard or Clear Metrics	9
Option A: Use the Dashboard Button	9
Option B: Remove the Docker Volume	9
Option C: Start With a Fresh Project Name	9
11. Run Commands	10
Secure Path	10
Signature Attack Path	10

Weak RSA Path	10
Availability Path	10
12. Security Takeaways	10
13. Quick Troubleshooting	10
Dashboard still shows old metrics	10
Dashboard does not update	10
Port already in use	11

1. Overview

This project is a local Java and Docker lab for secure and vulnerable RSA usage patterns.

The lab has four main paths:

1. A secure RSA encryption and signature flow.
2. A deliberately weak toy RSA encryption attack.
3. A JWT signature bypass caused by trusting `alg=none`.
4. A bounded local availability-load demo showing how expensive private-key operations need rate limits.

The attack scenarios are local simulations. They are not intended to attack any external system.

2. Files and Responsibilities

CryptoDemoApp.java

This is the HTTP service. It can run in two modes:

- `APP_MODE=secure`
- `APP_MODE=vulnerable`

Secure mode exposes:

- `/secure/public-key`
- `/secure/encrypt`
- `/secure/decrypt`
- `/secure/sign`
- `/secure/verify`
- `/token/issue`
- `/token/verify`

Vulnerable mode also exposes:

- `/toy/challenge`
- vulnerable behavior inside `/token/verify` for `alg=none`

AttackClient.java

This is the demo runner. It supports:

- `normal-secure`
- `signature-attack`
- `rsa-weak-key-attack`
- `ddos-demo`

It has local-target guardrails. By default it refuses targets outside localhost or Docker service names used by this lab.

DashboardApp.java

This is the browser dashboard. It tails the shared JSONL log file and charts:

- crypto latency
- HTTP status codes
- load-demo results
- raw JSON events

It also has **Reset Dashboard** and **Clear Metrics** actions that truncates the shared log file and resets the browser counters.

DemoLog.java

This helper writes every event to stdout and, if `LOG_FILE` is set, appends the same JSON line to a shared log file.

In Docker Compose, containers use:

```
LOG_FILE=/logs/events.jsonl
```

The dashboard reads from that file.

DemoJson.java

This is a tiny flat JSON helper. It keeps the project dependency-free for easy Java compilation inside Docker.

3. RSA Refresher

RSA is based on modular arithmetic.

At a high level:

1. Choose two large primes, p and q .
2. Compute $n = p * q$.
3. Compute $\phi = (p - 1) * (q - 1)$.
4. Choose public exponent e , commonly 65537.
5. Compute private exponent d , where:

$$e * d = 1 \bmod \phi$$

The public key is:

(n, e)

The private key is:

d

For textbook RSA encryption:

```
ciphertext = message^e mod n  
plaintext  = ciphertext^d mod n
```

For textbook RSA signatures:

```
signature = hash(message)^d mod n  
verify    = signature^e mod n
```

Real systems should not use textbook RSA directly. They use padding and encoding schemes such as:

- OAEP for encryption.
- PSS for signatures.

This lab uses both secure and insecure examples so the contrast is easy to explain.

4. Secure RSA in This Code

Key Generation

In `CryptoDemoApp`, startup creates an RSA key pair:

```
KeyPairGenerator generator = KeyPairGenerator.getInstance("RSA");
generator.initialize(keyBits, secureRandom);
this.rsaKeyPair = generator.generateKeyPair();
```

The default key size is 3072 bits:

`RSA_KEY_BITS=3072`

Secure mode refuses keys below 2048 bits:

```
if ("secure".equals(mode) && keyBits < 2048) {
    throw new IllegalArgumentException("secure mode requires RSA_KEY_BITS >= 2048");
}
```

This is an important mitigation. The toy attack works because the vulnerable RSA modulus is tiny. Secure RSA-2048 or RSA-3072 is not attacked in this lab.

Secure Encryption: RSA-OAEP-SHA256

The secure encryption endpoint uses:

```
Cipher.getInstance("RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
```

Despite the string containing ECB, RSA is not a block cipher mode here. This is the provider naming convention for RSA transformations in Java. The important part is:

`OAEPWithSHA-256AndMGF1Padding`

The code explicitly supplies OAEP parameters:

```
new OAEPParameterSpec(
    "SHA-256",
    "MGF1",
    MGF1ParameterSpec.SHA256,
    PSource.PSpecified.DEFAULT
)
```

OAEP matters because raw RSA is deterministic and structurally fragile. OAEP adds randomized padding and integrity checks around the message before RSA exponentiation.

Secure flow:

1. Client sends plaintext to `/secure/encrypt`.
2. Server encrypts using public key and OAEP.
3. Server returns Base64 ciphertext.
4. Client sends ciphertext to `/secure/decrypt`.
5. Server decrypts using private key and OAEP.
6. Server returns plaintext only if OAEP decoding succeeds.

The decrypt error is intentionally opaque:

```
json(exchange, 400, DemoJson.object("error", "decryption_failed"));
```

This avoids teaching a dangerous pattern where different decrypt failures leak useful oracle information.

Secure Signature: RSASSA-PSS-SHA256

The secure signature code uses:

```
Signature.getInstance("RSASSA-PSS");
```

With parameters:

```
new PSSParameterSpec(
    "SHA-256",
    "MGF1",
    MGF1ParameterSpec.SHA256,
    32,
    1
)
```

Meaning:

- Digest: SHA-256
- Mask generation: MGF1 with SHA-256
- Salt length: 32 bytes
- Trailer field: 1

Secure signing:

1. Server receives a payload.
2. Server signs it with the RSA private key using PSS.
3. Server returns the signature as Base64.

Secure verification:

1. Server receives payload and signature.
2. Server verifies with the RSA public key using the same PSS parameters.
3. Server returns `valid=true` or rejects the request.

5. Toy RSA: How the Weak RSA Attack Works

The vulnerable RSA demo is deliberately tiny and educational.

In `ToyRsa.create()`, the code uses:

```
p = 4,000,000,007
q = 4,000,000,063
e = 65,537
```

Then:

```
n = p * q
phi = (p - 1) * (q - 1)
d = e^-1 mod phi
```

Because `p` and `q` are only around 4 billion, `n` is only about 64 bits. That is small enough to factor quickly.

Challenge Creation

The vulnerable endpoint `/toy/challenge` takes a short secret:

```
TOY_SECRET=RSA42
```

It converts it to a positive integer:

```
BigInteger message = new BigInteger(1, secret.getBytes(StandardCharsets.UTF_8));
```

Then it performs textbook RSA:

```
BigInteger ciphertext = message.modPow(toyRsa.e, toyRsa.n);
```

The endpoint returns:

```
{
  "ciphertext": "...",

```

```

    "n": "...",
    "e": "65537"
}

```

This is intentionally unsafe because it publishes a tiny RSA modulus and uses no padding.

Attacker Calculation

The attacker sees public values:

```

n
e
ciphertext

```

The attacker factors n :

```

n = p * q

```

The code uses Pollard Rho:

```

BigInteger p = pollardRho(n);
BigInteger q = n.divide(p);

```

Once p and q are known, the attacker computes:

```

phi = (p - 1) * (q - 1)
d = e^-1 mod phi

```

Then decrypts:

```

plaintext = ciphertext^d mod n

```

In code:

```

BigInteger d = e.modInverse(p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE)));
String plaintext = new String(unsignedBytes(c.modPow(d, n)), StandardCharsets.UTF_8);

```

What This Demonstrates

This is not a claim that RSA-2048 can be factored during the demo. The lesson is:

- RSA security depends on key size.
- RSA encryption must use padding such as OAEP.
- Textbook RSA is not a safe application protocol.
- Small RSA keys fail because factoring n recovers the private key.

6. JWT and Signature Bypass

Normal JWT Structure

A JWT has three parts:

```

base64url(header).base64url(payload).base64url(signature)

```

The header usually declares the algorithm:

```

{"alg": "PS256", "typ": "JWT"}

```

The payload has claims:

```

{"sub": "alice", "role": "user", "iat": "..."}

```

The signature protects:

```

base64url(header) + "." + base64url(payload)

```

In secure mode, the code signs that input using RSASSA-PSS:

```
String signingInput = header + "." + payload;
String token = signingInput + "." + b64Url(signPss(signingInput.getBytes(StandardCharsets.US_ASCII)));
```

Secure Verification

The secure verifier extracts the header and checks:

```
if (!"PS256".equals(alg)) {
    reject
}
```

Then it verifies the signature with PSS:

```
verifyPss((parts[0] + "." + parts[1]).getBytes(StandardCharsets.US_ASCII),
    Base64.getUrlDecoder().decode(parts[2]))
```

The key security idea: the server pins the expected algorithm. It does not let the client choose whether a signature is required.

Vulnerable Bypass

In vulnerable mode, the token verifier has this teaching flaw:

```
if ("vulnerable".equals(mode) && "none".equalsIgnoreCase(alg)) {
    accept token
}
```

The attacker creates a token with:

```
{"alg":"none","typ":"JWT"}
```

And a payload:

```
{"sub":"mallory","role":"admin","iat":"..."}
```

Then the token is:

```
base64url(header).base64url(payload).
```

Notice the trailing dot and empty signature.

The vulnerable service accepts it because it trusts the token's algorithm header. The secure service rejects the same token because `alg=none` is not pinned to `PS256`.

Verification Policy

The bypass is not that JWT itself is bad. The problem is bad verification policy:

- The verifier accepted an unsigned token.
- The verifier treated a client-controlled header as security policy.
- The correct fix is to pin allowed algorithms server-side and verify the signature every time.

7. DDOS / Availability Load Demo

The DDOS demo is intentionally bounded and local. It demonstrates crypto cost exhaustion, not public-target DDOS.

RSA private-key operations are expensive. If a service exposes signing or decryption too freely, attackers can force the server to spend CPU on expensive operations.

In this lab:

1. The client asks the secure app to encrypt a small message.
2. It reuses the ciphertext.
3. It sends many decrypt requests concurrently.

4. The secure app allows requests until the rate limit is crossed.
5. It then returns HTTP 429.

Relevant settings in `docker-compose.ddos.yml`:

```
MAX_CRYPTOPS_PER_MIN=40
TOTAL_REQUESTS=90
CONCURRENCY=6
```

The rate limiter is deliberately simple:

```
return count.incrementAndGet() <= limit;
```

It logs:

```
secure_decrypt_ok
rate_limit_reject
availability_load_demo_complete
```

The final attacker summary includes:

```
status200
status429
otherStatus
elapsedMs
```

What This Demonstrates

The point is not that RSA alone causes DDOS. The broader lesson is:

- Private-key operations are expensive.
- Expensive endpoints need authentication, rate limits, and quotas.
- Decryption/signing should not be exposed as unauthenticated public utilities.
- Logs should clearly show success, rejection, latency, and limits.

8. Dashboard and Logs

All major actions emit JSON logs. Example fields:

```
ts
event
service
mode
requestId
algorithm
latencyMs
status
status200
status429
reason
```

The Docker containers write those logs to stdout and to:

```
/logs/events.jsonl
```

That path is backed by a Docker named volume:

```
demo-logs
```

The dashboard:

- serves HTML on port 8090
- streams logs from `/events`
- charts latency and status codes

- shows raw JSON logs in a scrollable pane
- can clear metrics through `/clear`

9. Why Old Metrics Reappear After Restart

Docker named volumes survive container restarts.

So if you run:

```
docker compose -f docker-compose.ddos.yml up secure-app dashboard
```

then stop and start again, the containers are new, but the named volume still contains:

```
/logs/events.jsonl
```

The dashboard loads previous lines from that file. That is why old metrics reappear.

10. How To Reset Dashboard or Clear Metrics

Option A: Use the Dashboard Button

Open:

```
http://localhost:8090
```

Click:

Reset Dashboard or Clear Metrics

This sends:

```
POST /clear
```

The dashboard truncates the JSONL file and resets the browser counters.

Option B: Remove the Docker Volume

Stop the stack and remove volumes:

```
docker compose -f docker-compose.ddos.yml down -v
```

For the secure stack:

```
docker compose -f docker-compose.secure.yml down -v
```

For the attack stack:

```
docker compose -f docker-compose.attack.yml down -v
```

Because the Compose project name comes from the directory, all three files usually refer to the same named volume:

```
i-want-to-demonstrate-rsa-encryption-demo-logs
```

You can also remove it directly:

```
docker volume rm i-want-to-demonstrate-rsa-encryption-demo-logs
```

Only do that when the demo containers are stopped.

Option C: Start With a Fresh Project Name

You can isolate a run with:

```
docker compose -p rsa-demo-fresh -f docker-compose.ddos.yml up --build secure-app dashboard
```

That creates a different volume name for that project.

11. Run Commands

Secure Path

```
docker compose -f docker-compose.secure.yml up --build secure-app dashboard
docker compose -f docker-compose.secure.yml --profile demo run --rm normal-demo
```

Signature Attack Path

```
docker compose -f docker-compose.attack.yml up --build secure-app vulnerable-app dashboard
docker compose -f docker-compose.attack.yml --profile attack run --rm signature-attack-demo
```

Weak RSA Path

```
docker compose -f docker-compose.attack.yml --profile attack run --rm weak-rsa-demo
```

Availability Path

```
docker compose -f docker-compose.ddos.yml up --build secure-app dashboard
docker compose -f docker-compose.ddos.yml --profile ddos run --rm bounded-load-demo
```

12. Security Takeaways

Use:

- RSA 2048+ minimum, 3072 where appropriate.
- OAEP for RSA encryption.
- PSS for RSA signatures.
- Strict server-side algorithm allowlists.
- Opaque decrypt errors.
- Rate limits and quotas around private-key operations.
- Clear logs for success, rejection, latency, and reason.

Avoid:

- Textbook RSA.
- Tiny RSA keys.
- Trusting JWT alg from clients.
- Accepting unsigned tokens.
- Unauthenticated decrypt/sign endpoints.
- Leaving expensive crypto operations unbounded.

13. Quick Troubleshooting

Dashboard still shows old metrics

Use the **Reset Dashboard** button in the top header, or the **Clear Metrics** button or run:

```
docker compose -f docker-compose.ddos.yml down -v
```

Then start again.

Dashboard does not update

Rebuild after code changes:

```
docker compose -f docker-compose.ddos.yml up --build secure-app dashboard
```

Port already in use

Ports used:

8080 secure app
8081 vulnerable app
8090 dashboard

Stop older containers:

```
docker compose -f docker-compose.attack.yml down  
docker compose -f docker-compose.ddos.yml down  
docker compose -f docker-compose.secure.yml down
```